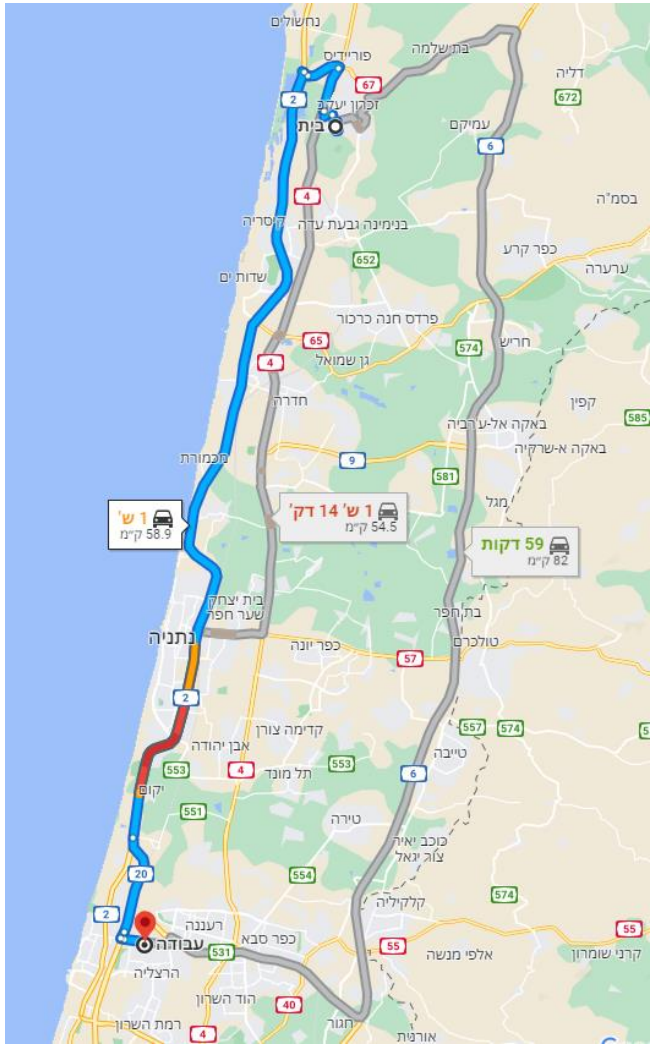
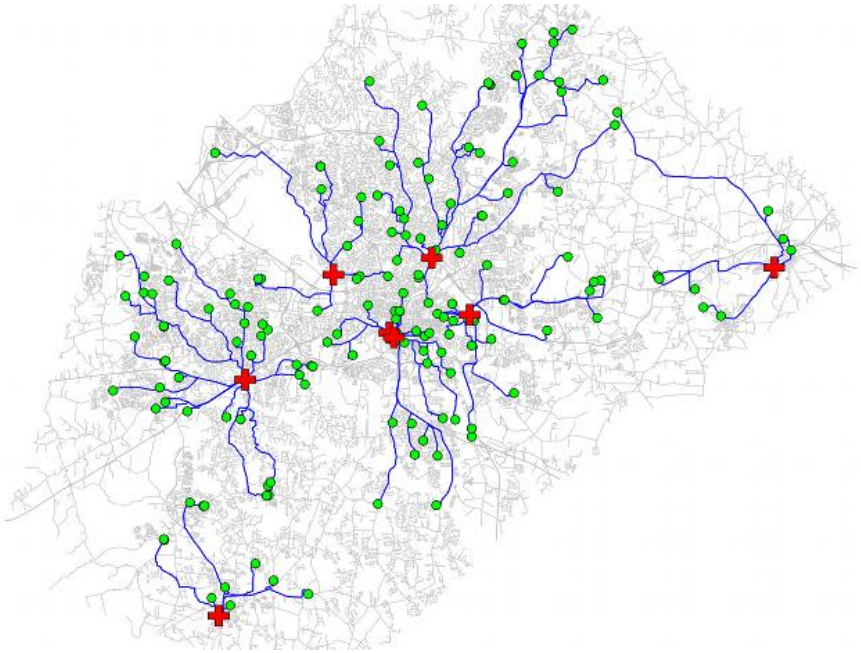


Algorithms

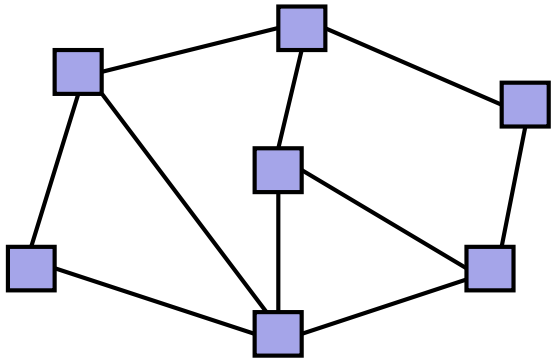
Shortest Path Algorithms

Reading: CLRS, Sections 24, 25.2

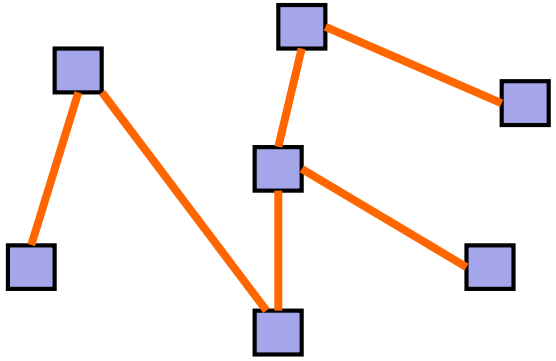


Review - Lecture 4

Spanning tree:
Input: An undirected connected graph G = (V,E).



Output: E’ $\subseteq$E such that (V,E’) is a tree



Review - Lecture 4

BFS Algorithm for finding a ST.

```
Enqueue(s) and mark s;
while (Q is not empty) do
    v ←Dequeue(Q);
    for each u adjacent to v do
        if u is not marked then
            add {v,u} to T;
            mark u;
            Enqueue(Q, u);
```

Time and space complexity: $O(|V|+|E|) = O(|E|)$
(analysis similar to DFS)

connected

Review - Lecture 4

BFS Algorithm for finding shortest paths from s.

The root s is labeled 0
Iteration i for i=0,1,…:
nodes that are labeled i mark their unlabeled neighbors i+1.

The collection of edges from labeled to unlabeled nodes form a directed tree, in which for every v, the (only) path from s to v is a shortest path.

```
Enqueue(s) and mark s;
while (Q is not empty) do
    v ←Dequeue(Q);
    for each u adjacent to v do
        if u is not marked then
            add {v,u} to T;
            mark u;
            label(u)=label(v)+1
            Enqueue(Q, u);
```

Discussion: BFS vs. DFS

Review - Lecture 4

Single source shortest path algorithms

- Using BFS, we solved the problem assuming all edges have unit length.
- A more practical setting: edges are not uniform.
- Length of path = sum of edge weights.

Input:

A directed graph G=(V,E),

A weight function w:E→R,

A source vertex s∈V.

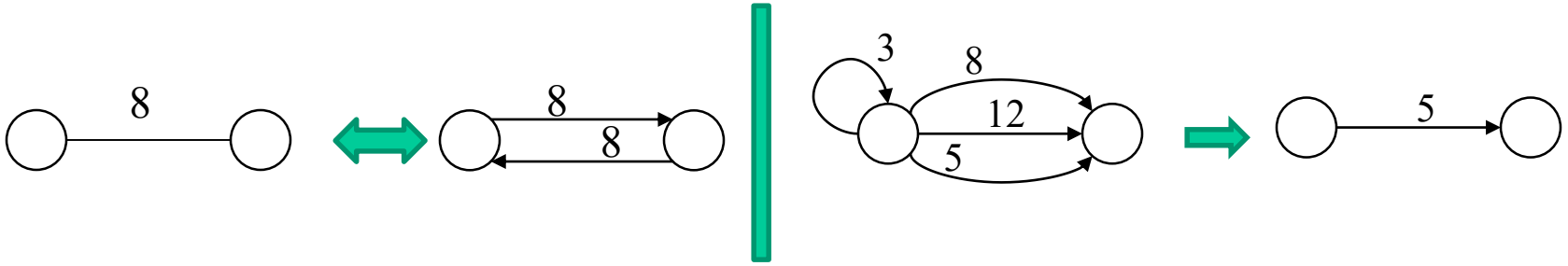
Output:

For each vertex v∈V, d(s,v) - its distance from s.

Review - Lecture 4

Single source shortest path algorithms

- We will assume w.l.o.g. that the graph is simple and directed.



For each node v, the algorithm keeps track of two variables:
v.d – length of shortest path detected so far
v.π - predecessor of v (parent).

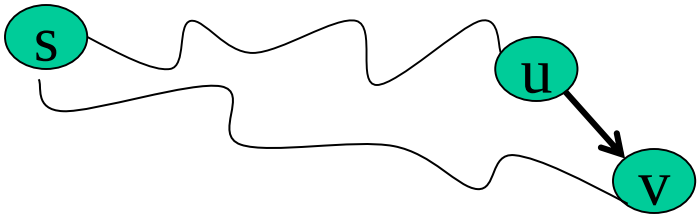
Review - Lecture 4

General structure of single source algorithms:

- v.d is initialized to infinity, except s.d = 0.
- The value of v.d may be updated (decreased) in relax operations.
- relax(u,v): v.d = min{v.d, u.d+w(u,v)}

current best

what we can achieve using (u,v)



Single-source Shortest Paths Problem

We will consider several classes of instances:

- ☑ Unit-weight graphs - done (BFS).
- ☐ Directed acyclic graphs.
- ☐ Graphs with non-negative weights.
- ☐ Graphs with arbitrary weights - (may be that no solution exists, in case of a negative cycle).

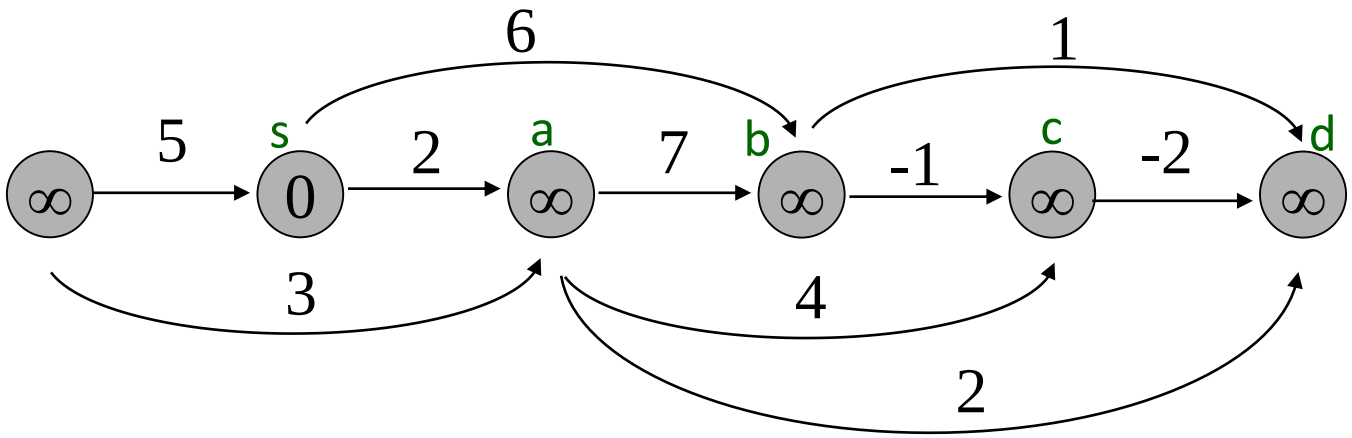
Directed Acyclic Graphs

Algorithm:

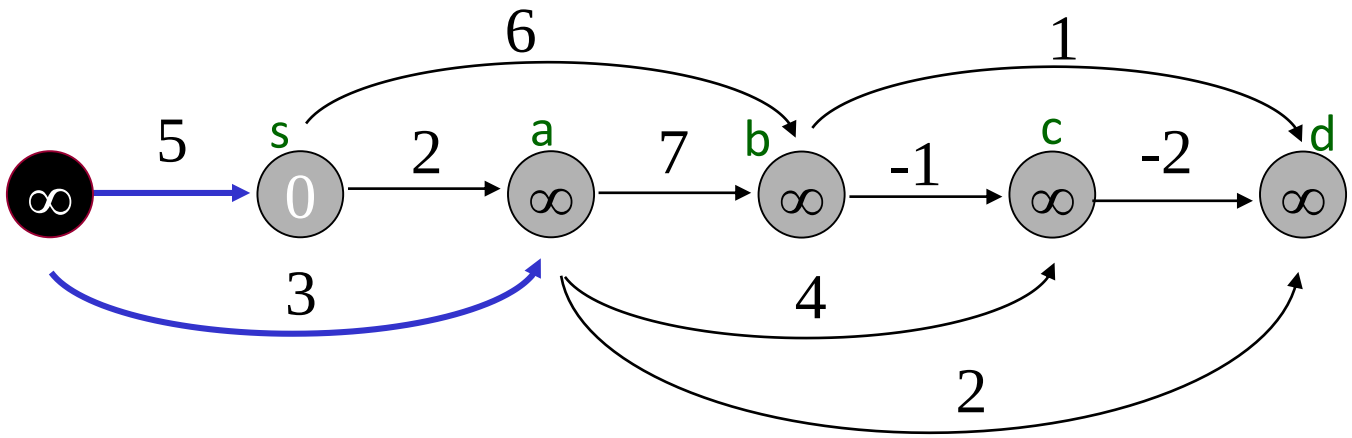
1. Topologically sort the vertices.
2. For each vertex u, in sorted order
   for each v such that $(u,v) \in E$
      relax(u,v)

relax all edges leaving u

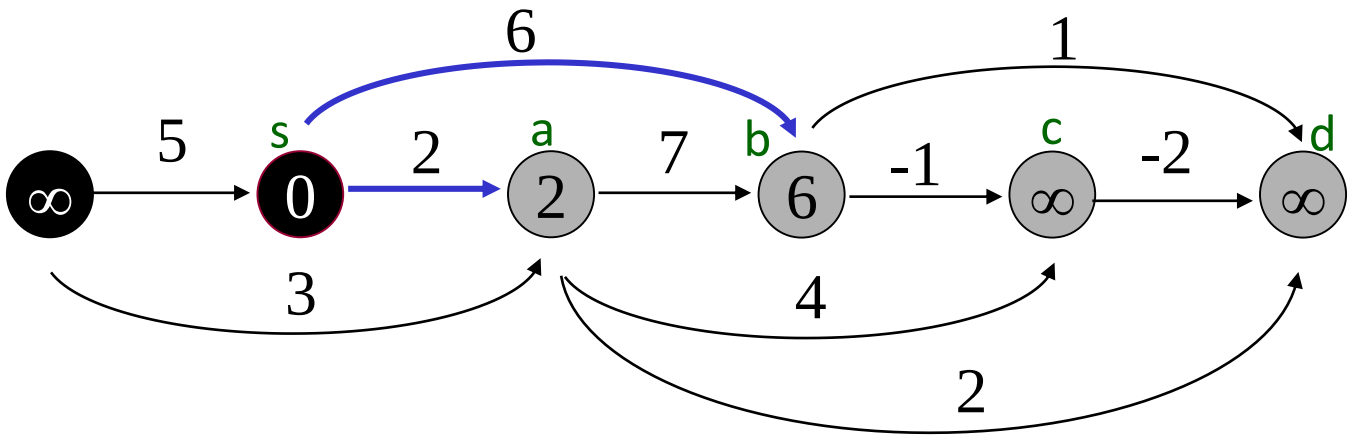
DAG shortest-paths Algorithm



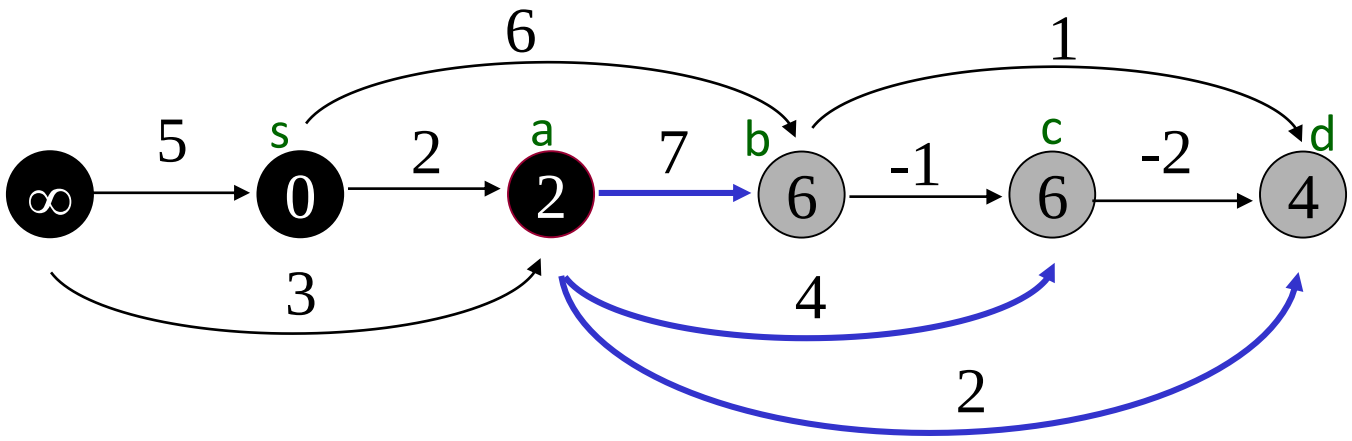
DAG shortest-paths Algorithm



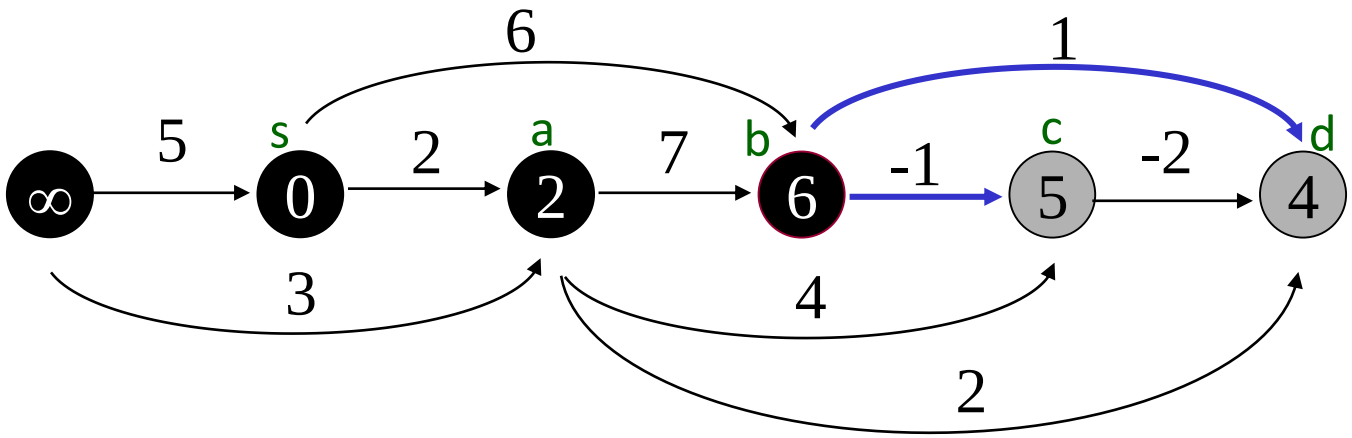
DAG shortest-paths Algorithm



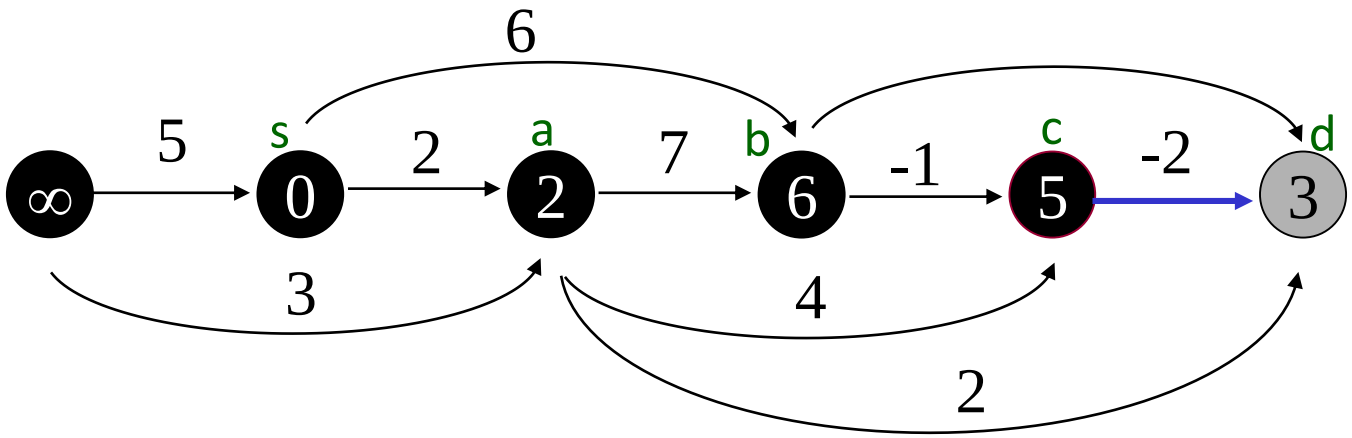
DAG shortest-paths Algorithm



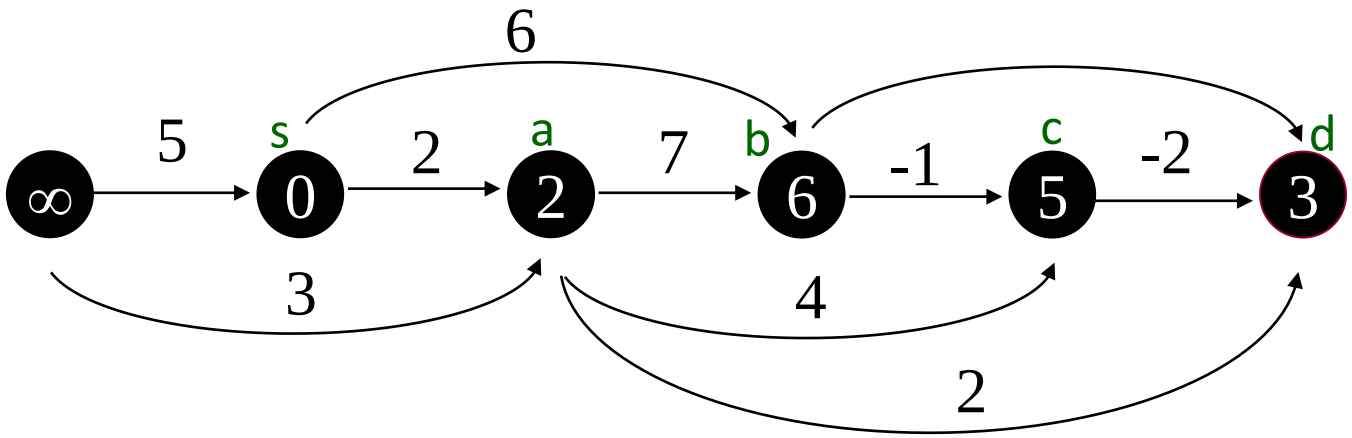
DAG shortest-paths Algorithm



DAG shortest-paths Algorithm



DAG shortest-paths Algorithm



Correctness

Theorem:

If G is acyclic then when the algorithm terminates, for each vertex v, v.d = d(s,v) holds.

Lemma:

When v is reached in the main loop, v.d = d(s,v) holds.

Proof:

1. Vertices that are not connected from s (before s in the topological sort, or after it, but not reachable): their distance is initialize to ∞ and it remains ∞ along the algorithm – as required.

Correctness (cont')

2. s and the vertices connected from s.

The proof is by induction on the order of the vertices in the topological sort.

Base: $d(s,s)=0$, s.d was initialized to 0 and is not updated during the algorithm.

Step: Let v be the next vertex in the toposort order.
Let $s\text{-}v_1\text{-}v_2\text{-}\ldots\text{-}v_k\text{-}v$ be some shortest path from s to v.
It must be that $s\text{-}v_1\text{-}v_2\text{-}\ldots\text{-}v_k$ is a shortest path from s to v_k.
By definition of toposort, v_k was processed already.
By the induction hypothesis, when v_k was processed it holds that $v_k.d=d(s,v_k)$.

In the iteration in which v_k is processed, relax $w(v_k,v)$ is performed and the distance of v is updated to be $d(s,v_k)+w(v_k,v)$, as required.

Time Analysis

- Each ‘relax’ operation takes O(1).

- Preprocessing: Topological sort takes O(n+m)
- Loop through n vertices: O(n)
- Total number of relax operations:

When vertex v is processed the number of relax operations is out_degree(v).

During the algorithm, each vertex is processed once, so the total number of relax operations is m and their total time complexity is O(m).

Total time : O(m+n)

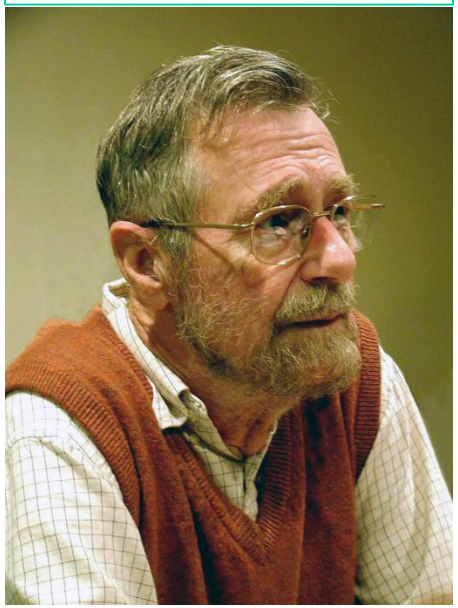
Graphs with non-negative weights

Q is a priority queue where priority is according to the value of d.

Dijkstra Algorithm

```
Q ← V(G)
while Q is not empty :
        u ← dequeue(Q)
        for all v such that (u,v)∈E :
                relax(u,v)
```

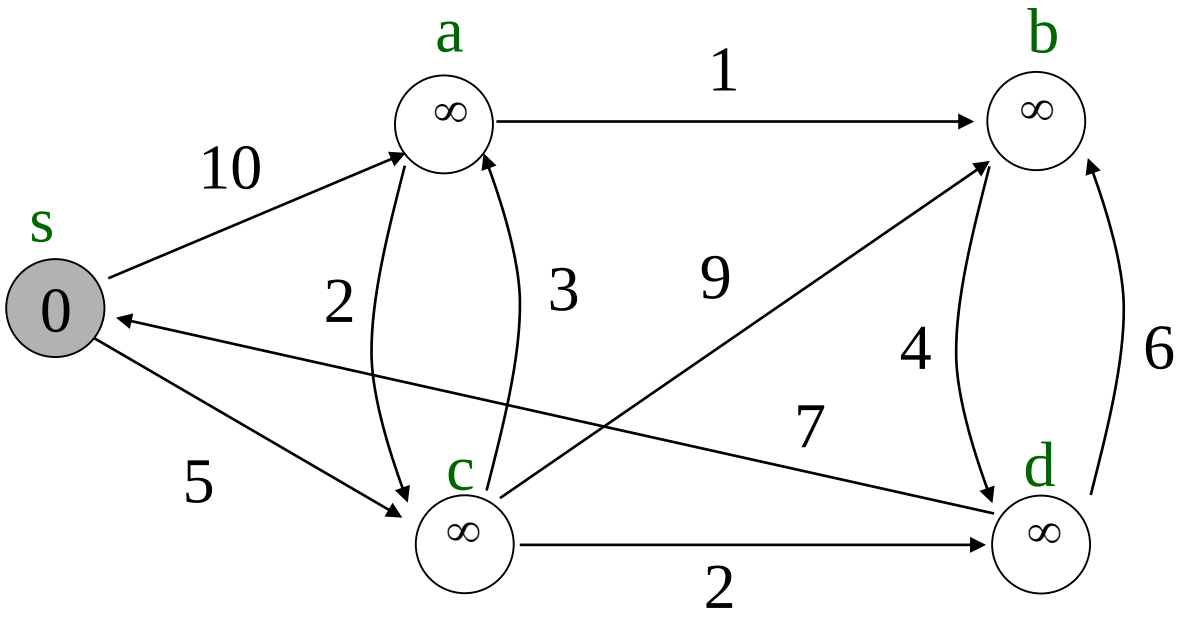
Note: The slides are in Dijkstra font!



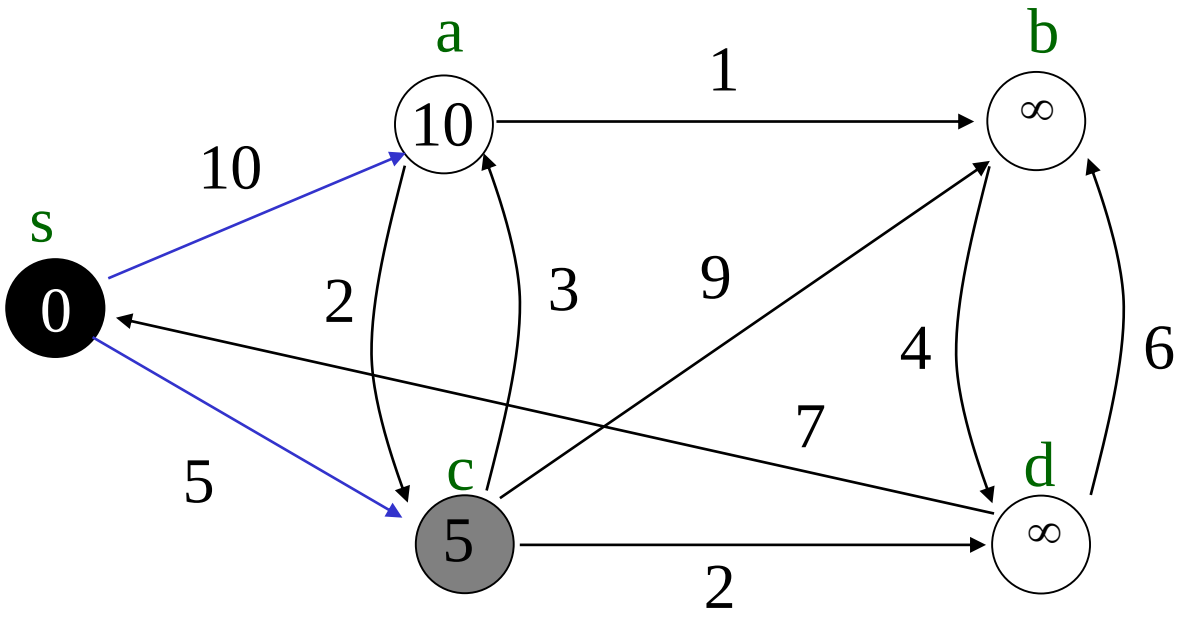
Dijkstra's Algorithm –an alternative description.

1. $s.d \leftarrow 0$ and for all $v \neq s$, $v.d \leftarrow \infty$.
2. $Q \leftarrow V$.
3. Let u be a vertex in Q for which $u.d$ is minimum.
4. for all v such that $(u,v) \in E$, perform relax(u,v)
5. $Q = Q - \{u\}$, if Q is not empty go to step 3.

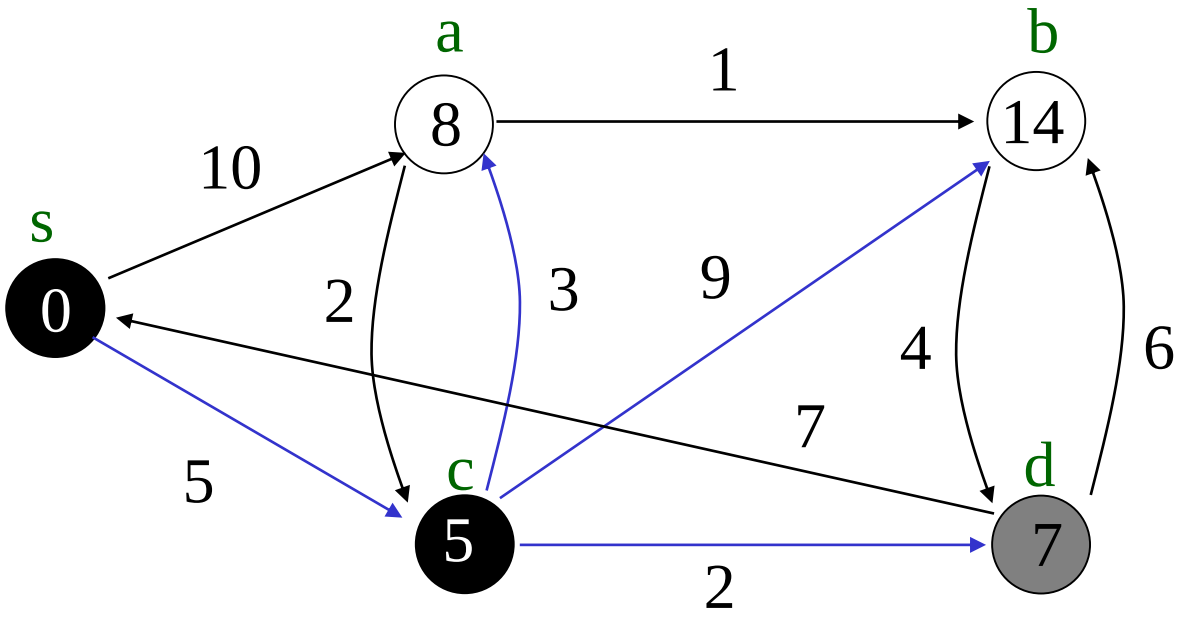
Dijkstra's Algorithm



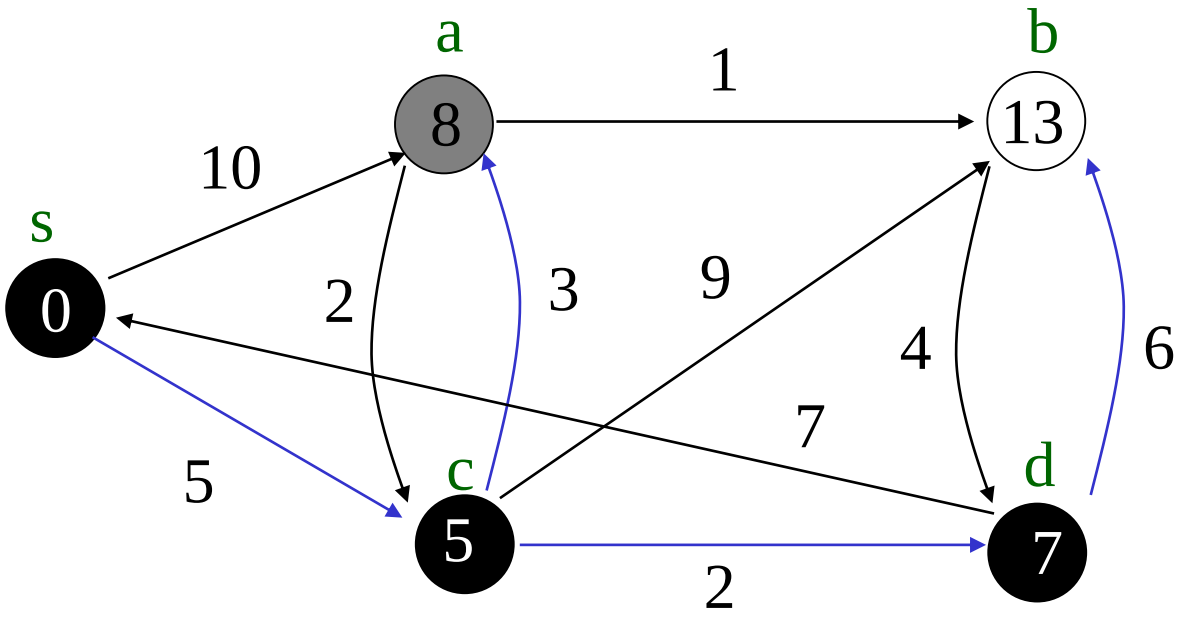
Dijkstra's Algorithm



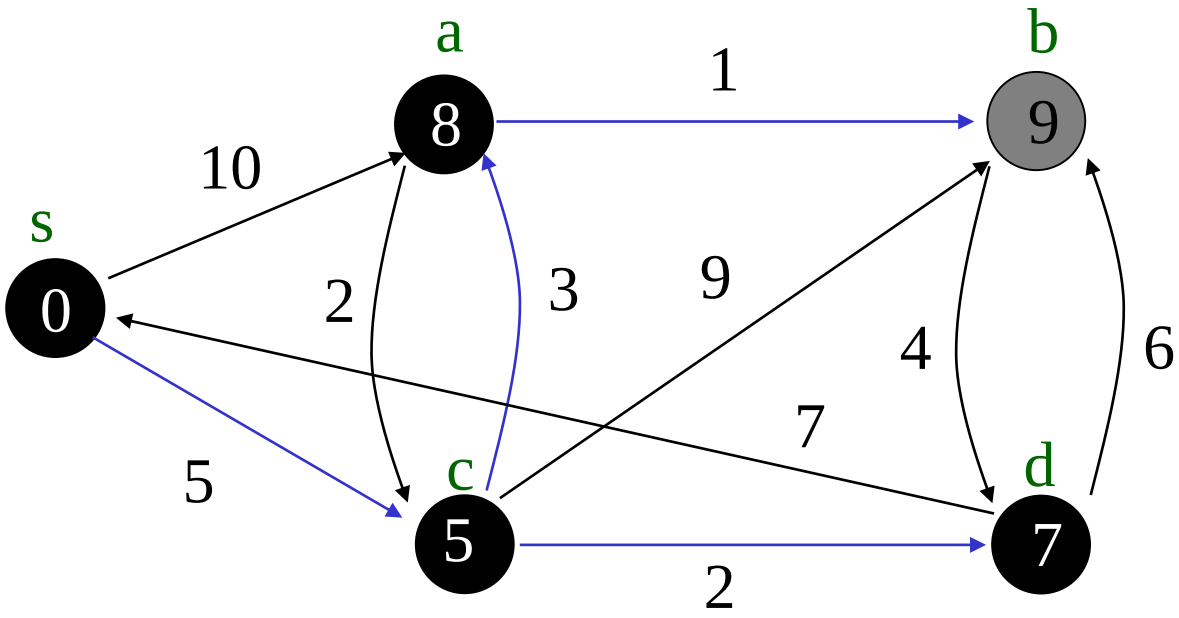
Dijkstra's Algorithm



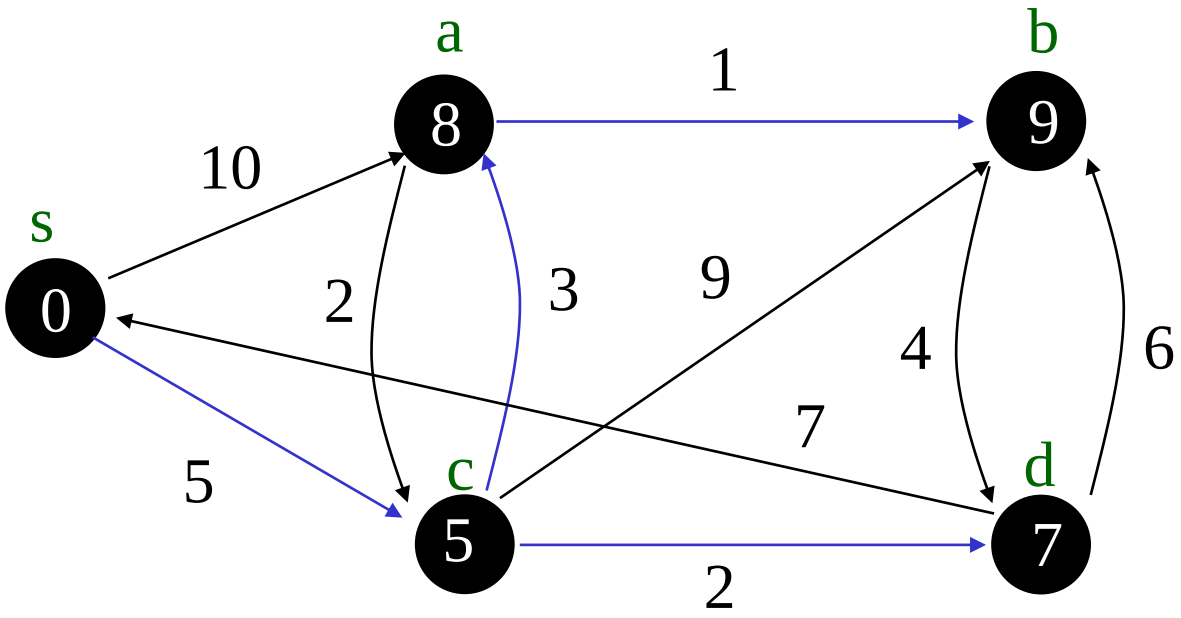
Dijkstra's Algorithm



Dijkstra's Algorithm



Dijkstra's Algorithm



Dijkstra Algorithm: Correctness

Theorem:

If all edges have non-negative weight then when Dijksrta terminates, for each vertex v v.d = d(s,v) holds.

Lemma: When a vertex v leaves the priority queue v.d = d(s,v) holds

Proof: In class

הוכחת נכונות – האלגוריתם של דייקסטרה.

טענה: כשהצומת v יוצא מ-Q מתקיים ש- v.d=d(s,v)

הוכחה באינדוקציה על סדר יציאת הצמתים מ-Q.

בסיס: s יוצא ראשון ואכן s.d=d(s,s)=0.

הוכחת נכונות – האלגוריתם של דייקסטרה.

צעד: נתייחס לאיטרציה שבה צומת v יוצא מהתור.

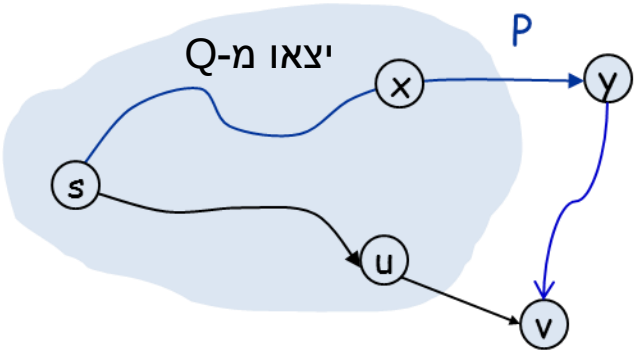
נניח שבשלב זה $v.\pi=u$. בהכרח u כבר יצא מהתור ובוצע relax(u,v). המסלול מ-s ל-u יחד עם הקשת (u,v), מגדירים מסלול באורך v.d ולכן $d(s,v) \le v.d$ (קיים מסלול באורך v.d).

הערה: טיעון זה תקף לכל אלגוריתם שמבוסס על ביצוע פעולות relax.

נראה שלא קיים מסלול קצר יותר.

הוכחת נכונות – האלגוריתם של דייקסטרה.

נניח בשלילה שקיים מסלול P מ-s ל- v כך ש- $w(P)<v.d$.
($w(P)$ מסמן את משקל המסלול P). יהי x הצומת האחרון ב-P שכבר יצא מ-Q. ודאי לא כל צמתי P יצאו מ-Q, אחרת v היה מסומן ב-$w(P)$. יהי y הצומת שאחרי x ב-P.



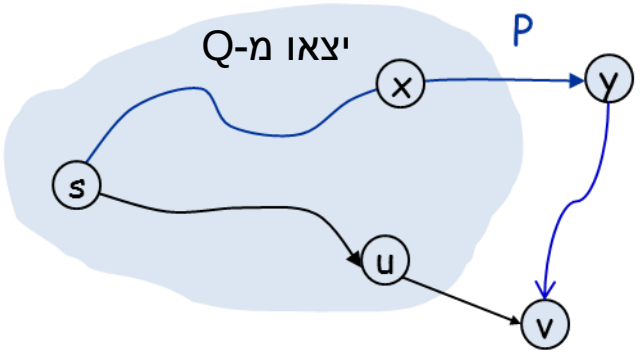
הוכחת נכונות – האלגוריתם של דייקסטרה.

בהכרח מתקיים: $y.d \geq d(s,x)+w(x,y) \geq w(P)$ (*)

מהנחת האינדוקציה, x כבר יצא מהתור ובוצע relax(x,y).

כי לתת-המסלול מ-y ל-v משקל אי שלילי

האלגוריתם בחר להוציא את v מ-Q, ולכן $y.d \leq v.d$.
יחד עם (*) נקבל $w(P) \leq v.d$ בסתירה לכך ש- $w(P) < v.d$.



Correctness Proof – Dijkstra Algorithm.

Claim: When a vertex v leaves Q it holds that v.d=d(s,v).

Proof: By induction on the order according to which the vertices leave Q.

Base: s is the first to leave Q and indeed s.d=d(s,s).

Correctness Proof – Dijkstra Algorithm.

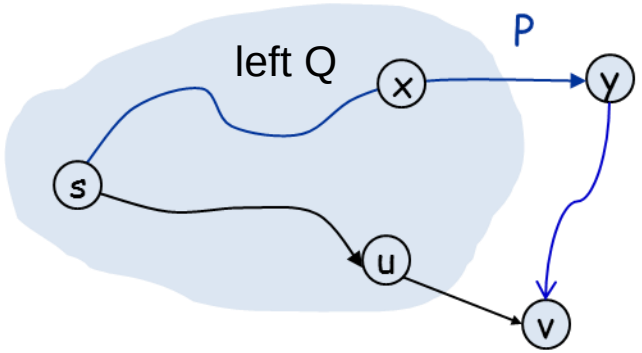
Step: Consider the iteration in which v leaves Q. Assume that at this time point $v.\pi = u$.
The path to u together with the edge (u,v) implies that there exists a path from s to v whose length is v.d.
Therefore $d(s,v) \leq v.d$.

Note: This is true in general for every application of the relax method.

We show that no shorter path exists.

Correctness Proof – Dijkstra Algorithm.

Assume, toward a contradiction, that there is a path P from s to v whose weight satisfies w(P)<v.d.
Let x be the last vertex on P that has already been removed from Q. Note that not all vertices of P can have left Q, since in that case v would already have been assigned the value w(P).
Let y be the successor of x on the path P.



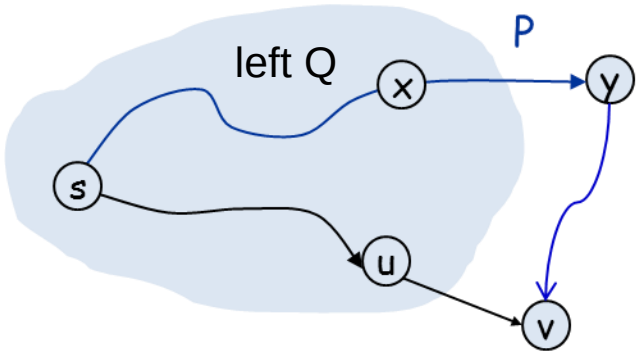
Correctness Proof – Dijkstra Algorithm.

We have $w(P) \geq d(s,x)+w(x,y) \geq y.d$ $(*)$

Since the sub-path y-v has non-negative weight.

Since x already left Q, and relax(x,y) was performed.

Since Dijkstra chose to consider v (before considering y) it must be that $v.d \leq y.d$.
Therefore, by $(*)$, $v.d \leq w(P)$ contradicting our assumption that $w(P)<v.d$.



Running Time of Dijkstra's Algorithm Implementation 1: binary heap

Data structure:

- Adjacency lists.
- Binary heap for the 'd' values.

Each relax operation takes O(log n).

- initialization: O(n) time
- n deleteMins: O(n log n) time
- m relax operations: O(m log n) time

In each iteration we check out_degree(u) values.
The total sum of out-degrees is m.

- Running time: O((n + m) log n))

Running Time of Dijkstra's Algorithm Implementation 2: array

Data structure:

- Adjacency lists.
- An array for the 'd' values.

Complexity:

In each iteration:

1. Finding a vertex u in Q with minimal $u.d$.
   In the whole execution: $O(n^2)$
2. m relax operations, each takes $O(1) \rightarrow O(m)$

All together: $O(m+n^2) = O(n^2)$ (simple graph, so $m \leq n(n-1)$)

Running Time of Dijkstra's Algorithm Implementation 3: Fibonacci heaps

Data structure:

- Adjacency lists.
- Fibonacci heap for the 'd' values.

Complexity: $O(m + n \log n)$

Fibonacci heap: any sequence of x insert/decrease-key operations and y delete operations takes $O(x + y \log n)$ worst case time, where n is the maximum heap size.

Dijktra's Algorithm - Discussion

- Why does the algorithm fail when the graph includes negative weights?

Graphs with Arbitrary Weights

Input: A simple directed graph G=(V,E),
a weight function w: E→ R, a source node v∈V.

- w(u,v) may be negative.
- Assume first that there are no negative cycles.

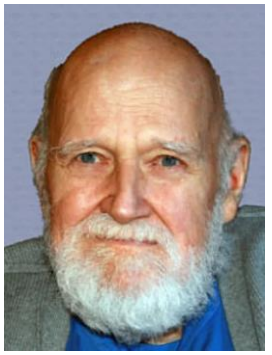
Bellman-Ford Algorithm:

repeat n-1 times:

    for all edges (u,v):

        relax(u,v)





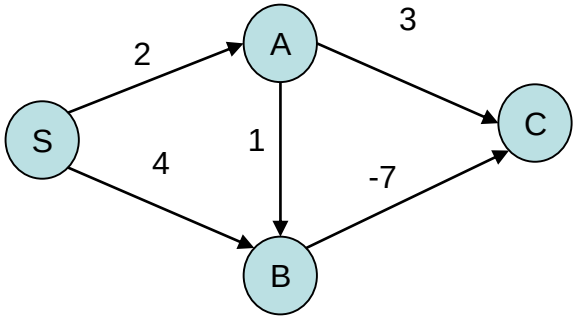
Bellman-Ford Algorithm

How do we implement this algorithm?

- Order the edges: $e_1, e_2, \ldots, e_{|E|}$. *(arbitrary order)*
- Every iteration of the repeat loop is denoted 'a sweep'.
- Every sweep consists of $|E|$ relax operations: first relaxing e_1, then e_2, etc.,
- Perform additional sweeps until an entire sweep produces no updates (might be that less than $|V|-1$ sweeps are needed).

Bellman-Ford algorithm

- Running example:



- Edge's order: SB, BC, AB, SA, AC.

Bellman-Ford - Correctness

Theorem: If G has no negative cycles then when the algorithm terminates, for each v, v.d = d(s,v) holds.

Lemma: If the shortest path from s to v consists of k edges, then after k repetitions of the main loop, v.d = d(s,v) holds.

Proof: next slide.

By the lemma, since k is bounded by n-1 (remember, no negative cycles), there are at most n-1 edges in a shortest path.

הוכחת נכונות – בלמן-פורד

יהי v צומת קשיר מ-s ויהי $s=v_0-v_1-v_2-\ldots-v_{k-1}-v_k=v$ מק"ב מ-s ל-v. אין מעגלים שליליים ולכן המסלול פשוט.

נוכיח באינדוקציה על i שלכל $i=0,\ldots,k$ מתקיים $v_i.d=d(s,v_i)$ אחרי ה-sweep ה-i.

בסיס: $i=0$. באיתחול $s.d=0=d(s,s)$ ולא יעודכן בהמשך.

צעד: מהנחת האינדוקציה, $v_{i-1}.d=d(s,v_{i-1})$ אחרי ה-sweep ה-i-1. במהלך ה-sweep ה-i יבוצע $relax(v_{i-1},v_i)$ ולכן ערך $v_i.d$ יתעדכן להיות $d(s,v_{i-1})+w(v_{i-1},v_i)$.

זהו אכן אורך המק"ב מ-s ל-v.

Correctness Proof – Bellman -Ford

Let v be a node reachable from s and let $s=v_0-v_1-v_2-\ldots-v_{k-1}-v_k=v$ be a SP from s to v. There are no negative cycles, therefore the path is simple.

We prove by induction on i that for all i=0,…,k it holds that $v_i.d=d(s,v_i)$ after the i-th sweep.

Base: i=0. We initialize $s.d=0=d(s,s)$, and s.d is not updated anymore.

Step: By the induction hypothesis $v_{i-1}.d=d(s,v_{i-1})$ after the (i-1)th sweep. During the i-th sweep, relax(v_{i-1},v_i) is performed and the value of v_i.d is updated to be $d(s,v_{i-1})+w(v_{i-1},v_i)$. This is indeed the length of the SP from s to v.

BF algorithm – run time analysis

- $|E|$ relax operations in every sweep.
- $|E| \cdot (|V|-1)$ relax operations in total.
- Each relax takes $O(1)$ –only need an array with the $v.d$ values, and direct access to the edge weights (can be kept in the graph's data structure).

→ The time complexity of BF is $O(|E| \cdot |V|)$.

Detecting Negative cycles

- Using BF to detect negative cycles:
- Theorem: The graph includes a negative cycle if and only if for some vertex v, v.d is reduced in sweep number |V|.
- Practically: After |V|-1 sweeps perform:
- for each (u,v)$\in$E
  - If v.d > u.d + w(u,v) then print "A negative cycle exists"
- Proof: In class + Moodle.

All-pair Shortest Path

- Input: a directed graph G=(V,E).

  A weight function w: E→ R

  w(u,v) may be negative.
- Let $V = \{1, 2, ..., n\}$.
- Output: All-pair shortest path: for any two vertices v,u in V, what is the shortest path from v to u
  - we will only be interested in the *length* of that path.

All-pair Shortest Path

- We can solve this problem using single-source shortest path algorithms. For example, we can run Bellman-Ford |V| times (one time for each possible selection of the source vertex s).
- Time complexity: $|V|*O(|V||E|)=O(|V|^2|E|)$
- We will see a solution based on Dynamic Programming.

All-pair Shortest Path

Define

$$\delta^0(i,j) = \begin{cases} w(e) & \text{if } (i,j) \in E, \\ \infty & \text{if } (i,j) \notin E. \\ 0 & \text{if } i = j \end{cases}$$

Let $\delta^k(i,j)$ be the length of a shortest path from i to j among all paths which may pass through vertices 1,2,...,*k* but do not pass through vertices *k+1, k+2,...,n.*

Floyd -Warshall Algorithm (1962)

1. Init $\delta^0(i, j)$ as defined earlier
2. $k \leftarrow 1$
3. For every $1 \leq i, j \leq n$ compute

$$\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j) \}.$$

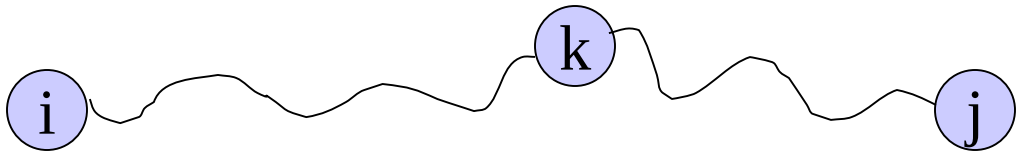
4. If $k = n$, stop. If not, increment k and go to step 3.

Floyd -Warshall Algorithm

$$\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j)\}.$$

The shortest path from i to j which may pass through vertices 1,2,...,*k* but do not pass through vertices *k+1, k+2,...,n:*

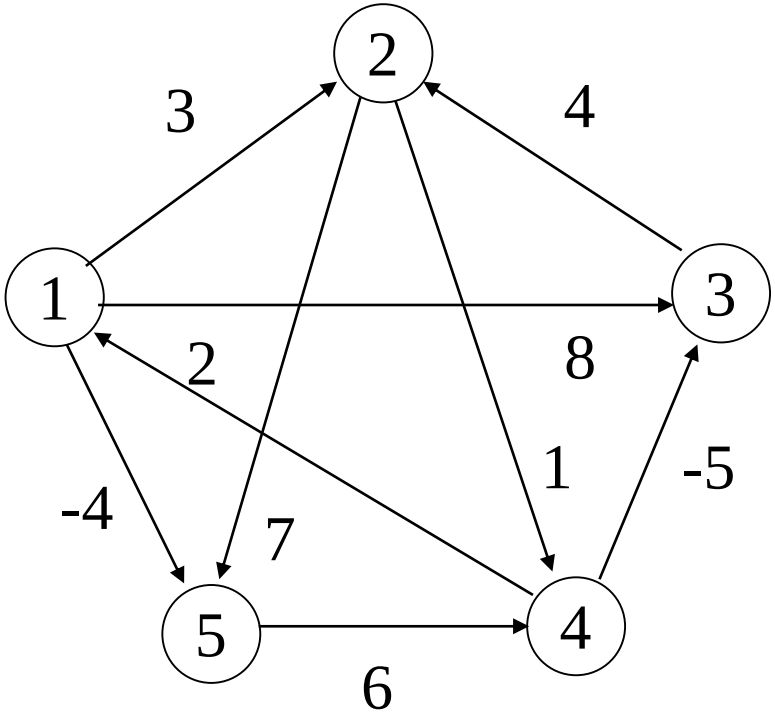
1. Might not pass through vertex k, or
2. Might pass through k, and then it is composed by two already-computed shortest paths.



Floyd -Warshall Algorithm

δ^0

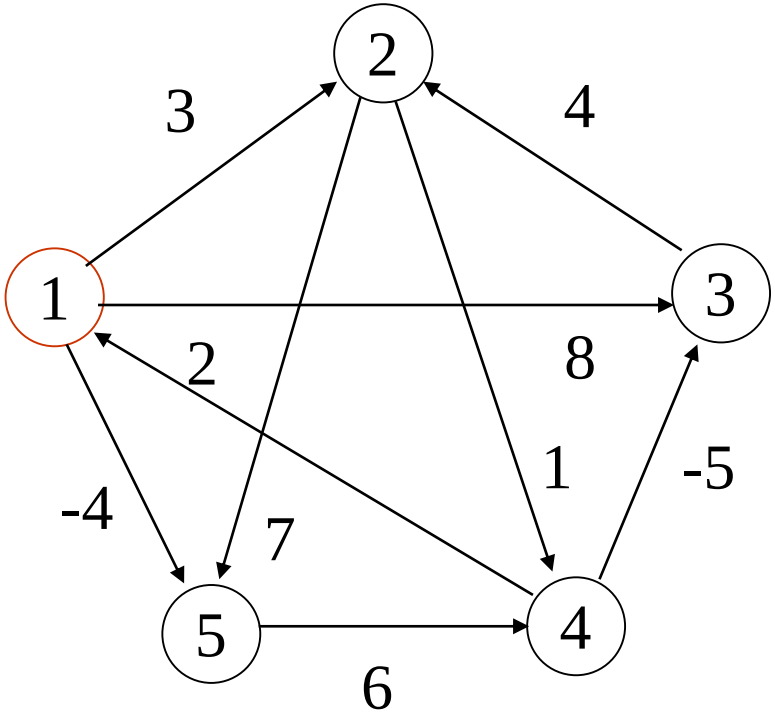
	1	2	3	4	5
1	0	3	8	--	-4
2	--	0	--	1	7
3	--	4	0	--	--
4	2	--	-5	0	--
5	--	--	--	6	0



Floyd-Warshall Algorithm

δ^1

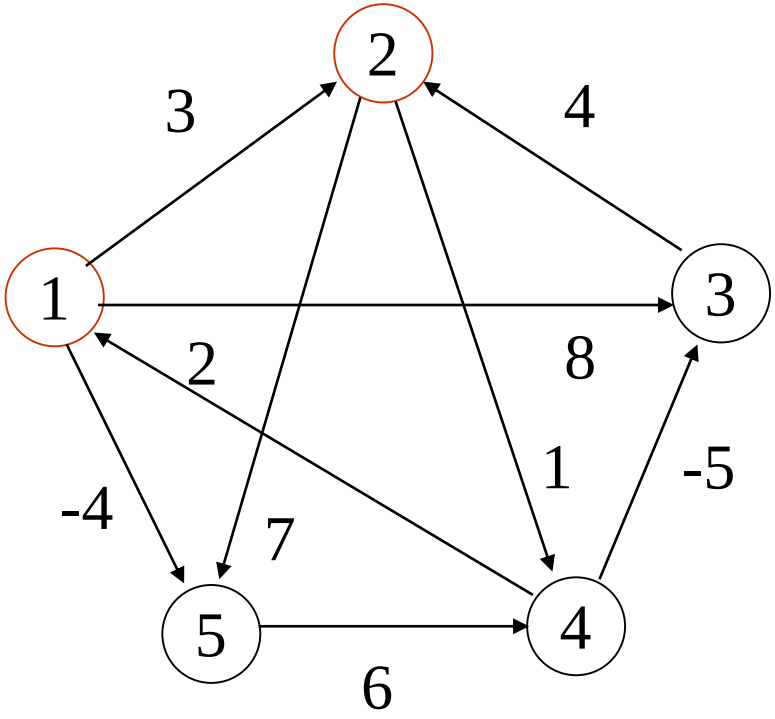
	1	2	3	4	5
1	0	3	8	--	-4
2	--	0	--	1	7
3	--	4	0	--	--
4	2	5	-5	0	-2
5	--	--	--	6	0



Floyd -Warshall Algorithm

δ^2

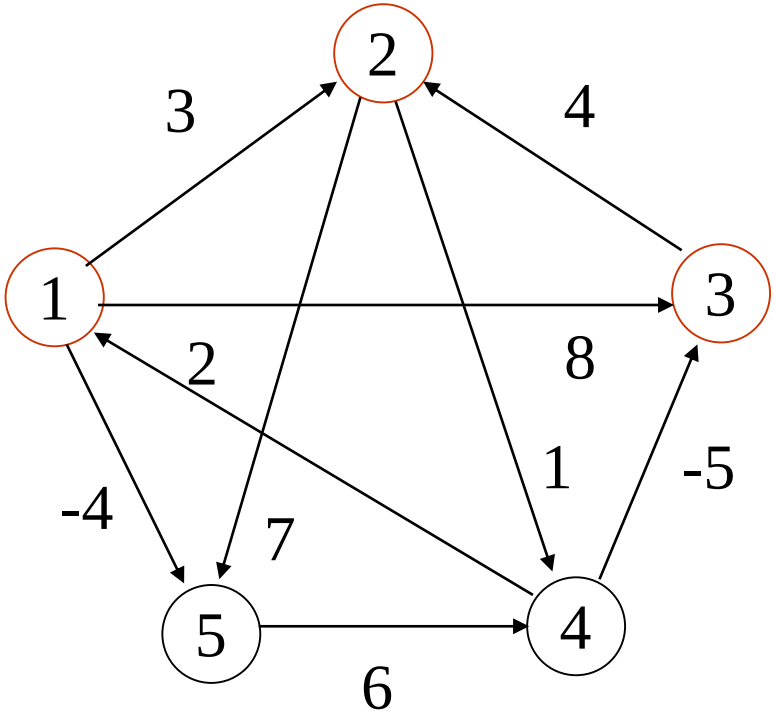
	1	2	3	4	5
1	0	3	8	4	-4
2	--	0	--	1	7
3	--	4	0	5	11
4	2	5	-5	0	-2
5	--	--	--	6	0



Floyd -Warshall Algorithm

δ^3

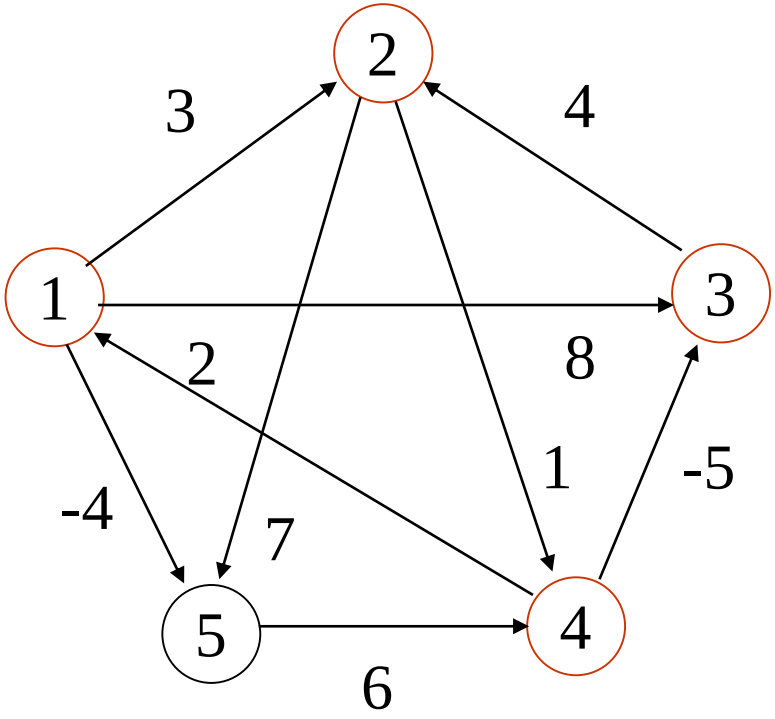
	1	2	3	4	5
1	0	3	8	4	-4
2	--	0	--	1	7
3	--	4	0	5	11
4	2	-1	-5	0	-2
5	--	--	--	6	0



Floyd -Warshall Algorithm

δ^4

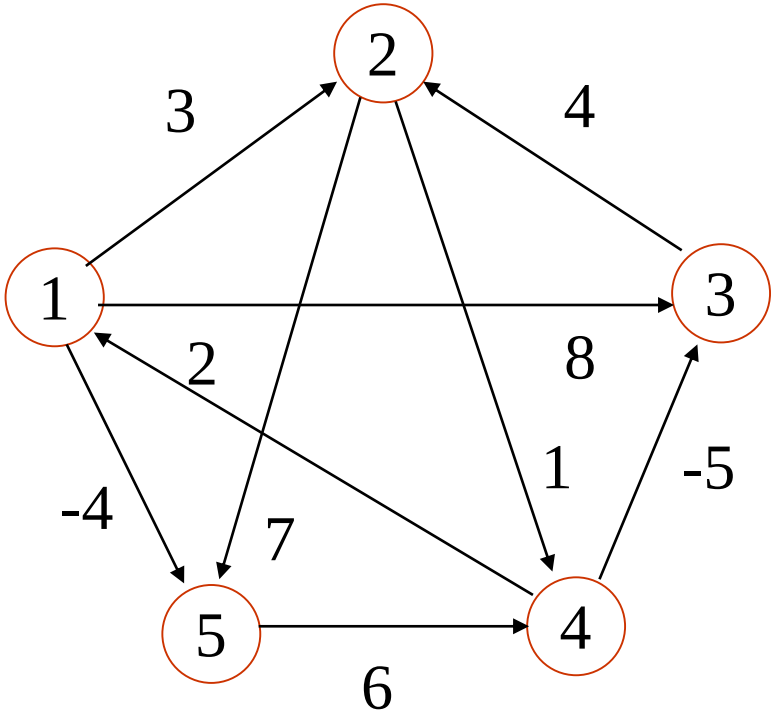
	1	2	3	4	5
1	0	3	-1	4	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0



Floyd -Warshall Algorithm

δ^5

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0



Floyd -Warshall Algorithm

$$\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j)\}.$$

Theorem: The value of $\delta^n(i, j)$ is the shortest path from i to j

Proof idea: By induction on k, the value of $\delta^k(i, j)$ is correct. In particular, for $\delta^n(i, j)$ we get the shortest path.

Floyd -Warshall Algorithm

Time complexity:

Each application of step 3 requires $O(n^2)$ operations, and step 3 is repeated n times. Thus, the total time complexity is $O(n^3)$.

Negative cycles detection:

If a negative cycle exists, then for some i and k, we will get $\delta^k(i, i) < 0$.

Shortest-path algorithms - Summary

- Single source, no weights: BFS - O(n+m)
- Single source – DAG: O(n+m)
- Single source, non-negative weights: Dijkstra O((n + m) log n)) or $O(n^2)$ or (m+n log n).
- Single source, arbitrary weights:

  Bellman-Ford: O(nm)
- All-pair shortest path, arbitrary weights: Floyd-Warshall : $O(n^3)$